



nextwork.org

Create Kubernetes Manifests



yakubu abdullahi yusuf

```
apiVersion: v1
kind: Service
metadata:
  name: nextwork-flask-backend
spec:
  selector:
    app: nextwork-flask-backend
  type: NodePort
  ports:
    - port: 8080
      targetPort: 8080
      protocol: TCP
```



yakubu abdullahi yusuf

NextWork Student

nextwork.org

Introducing Today's Project!

In this project, I will set up a Kubernetes Deployment manifest to define how the backend application is deployed in the cluster. I will also create a Service manifest to make the backend accessible to users. .

Tools and concepts

I used Amazon EKS, Git, ECS, Docker, GitHub, EC2 instances to I used Amazon EKS, Git, ECS, Docker, GitHub, and EC2 instances to push and manage the backend application in a cloud environment. These tools allowed me to version control the code, containerize the application, store and retrieve Docker images and also enabled scalable deployment and management of the application.

Project reflection

I chose to do this project today to gain more hands-on experience with Kubernetes and cloud deployment.

This project took me approximately twenty minutes to complete.



Manifest files

kubernetes manifest are set of instructions that tells Kubernetes how to run applications. Kubernetes uses it to know what your app needs, like which containers to run, how many copies to create, and how much memory to allocate.

A Kubernetes Deployment manages how an application is deployed and maintained within the cluster. The container image URL in my Deployment manifest tells Kubernetes which container image to pull and run when creating the application pods.

```
GNU nano 2.9.3 flask-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nextwork-flask-backend
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nextwork-flask-backend
  template:
    metadata:
      labels:
        app: nextwork-flask-backend
    spec:
      containers:
        - name: nextwork-flask-backend
          image: 176971016201.dkr.ecr.us-east-2.amazonaws.com/nextwork-flask-backend-ecr:latest
          ports:
            - containerPort: 8080
```



yakubu abdullahi yusuf

NextWork Student

nextwork.org

Service Manifest

A Kubernetes Service exposes an application by providing a stable way to access it within or outside the cluster. You need a Service manifest to define how traffic is routed to the application's pods and to ensure consistent access even when pods are created or replaced.

My Service manifest sets up a NodePort Service that exposes the backend application running on port 8080. It uses a selector to connect the Service to pods labeled nextwork-flask-backend and allows external traffic to reach the application through the node's IP address using the TCP protocol.



yakubu abdullahi yusuf

NextWork Student

nextwork.org

```
apiVersion: v1
kind: Service
metadata:
  name: nextwork-flask-backend
spec:
  selector:
    app: nextwork-flask-backend
  type: NodePort
  ports:
    - port: 8080
      targetPort: 8080
      protocol: TCP
```

```
~
~
~
~
~
~
~
~
~
~
```



yakubu abdullahi yusuf

NextWork Student

nextwork.org

Deployment Manifest

Annotating my Deployment manifest helped me better understand the use of each configuration setting and how the different components work together.

A notable line in the Deployment manifest is the number of replicas, which specifies how many identical copies of the application should be running at the same time. Pods are relevant to this because each replica represents a pod that runs an instance of the application. By increasing the number of replicas, Kubernetes can improve availability and handle more traffic by distributing the workload across multiple pods.

There isn't anything unclear about the manifest.



yakubu abdullahi yusuf

NextWork Student

nextwork.org

```
GNU nano 8.3 flask-deployment.yaml
apiVersion: apps/v1 #Specifies the API version for this Deployment
kind: Deployment #This is a Deployment object
metadata: #holds the details about the resource
  name: nextwork-flask-backend
  namespace: default
spec:
  replicas: 3
  selector: # Matches pods with this label so the Service knows which pods to send traffic to
    matchLabels:
      app: nextwork-flask-backend
  template: # Label applied to pods created by this Deployment
    metadata:
      labels:
        app: nextwork-flask-backend
    spec:
      containers:
        - name: nextwork-flask-backend # Name of the container running inside the pod
          image: 176971016201.dkr.ecr.us-east-2.amazonaws.com/nextwork-flask-backend-ecr:latest # Specifies which container image Kubernetes should pull and run
          ports:
            - containerPort: 8080 # Port on which the application inside the container listens
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

